

Programación con Objetos 2 - UNQ
1er. Semestre 2022 - Parcial 1ra. Fecha - Parte Práctica

Compañía de Seguros

Una compañía de seguros vende pólizas para asegurar los bienes de sus clientes. De esta manera, una póliza de seguros especifica un inventario que consta de un conjunto de ítems, cada uno de los cuales, a su vez, especifica una cantidad y un valor monetario por unidad. El valor de un ítem se define por la multiplicación de estos dos valores (ej: un ítem de 4 sillas a \$2000 c/u tiene un valor de \$8000).

Además de un inventario, a una póliza puede consultársele en cualquier momento:

- El monto asegurado: es la sumatoria del valor de todos sus ítems
- El precio: es el precio que el usuario debe pagar mensualmente por la misma y se calcula como el 0,75% del monto asegurado, más Gastos Administrativos.
- Gastos Administrativos: a una póliza pueden agregársele Gastos Administrativos, cada uno especifica simplemente un concepto y un monto.

La póliza pasa por varias fases:

Abierta: Inicialmente se encuentra abierta, y sobre la misma tienen sentido las siguientes operaciones:

- Agregar Ítem: permite agregar un ítem al inventario de la póliza. Es una operación que se espera que se invoque varias veces.
- Cerrar Inventario: indica que se ha finalizado la carga del inventario, y la póliza pasa a estar cerrada.

Cerrada: Estando Cerrada, se pueden invocar las siguientes operaciones de la misma:

- Agregar Ítem: permite agregar un ítem a la póliza, pero se cargará un Gasto Administrativo en concepto de "Recargo por extensión" del 3% del valor del ítem por cada uno que se agregue.
- Pagar: permite efectuar el pago de la póliza, lo cual la convierte en una póliza vigente (la póliza entra en vigencia ni bien se abona).

Vigente: En una póliza vigente el agregado de nuevos ítems no tiene ningún efecto (no los agrega). Si aún así se desea hacerlo, se debe ejecutar la siguiente operación:

- Cancelar: Elimina todo
- s los costos administrativos existentes y pasa la póliza a "Abierta", conservando el inventario actual y permitiendo al usuario reanudar el proceso a partir del mismo.

Las operaciones que no se mencionan en las fases anteriores, simplemente no surten ningún efecto en caso de invocarse.

Bonificaciones

A una póliza se le puede aplicar una bonificación a partir de un código de promoción único que se especifica en el momento de aplicar la misma. El proceso de aplicación de bonificación es el siguiente:

- 1.- Chequear validez del código de bonificación recibido
- 2.- Aplicar el descuento.
- 3.- Anular el código (para que no se vuelva a utilizar)
- 4.- Enviar un mail de notificación al titular de la póliza.

Los pasos 1, 3 y 4 son siempre los mismos para todos los casos, y para ejecutarlos debe simplemente servirse de la siguiente interfaz (no debe implementar sus mensajes):

```
public interface BonificacionService{

    /* Devuelve true si el código recibido es válido y está vigente */
    public boolean códigoVálido(int código);

    /* Anula el código recibido para que no se pueda re-utilizar */
    public void anularCódigo(int código);

    /* Envía un mail a la dirección recibida comunicando que se aplicó una
       bonificación con el código recibido. */
    public void notificarTitular (String mailTitular, int código);
}
```

Sin embargo, el paso 2 depende de la fase en que se encuentre la póliza, de la siguiente manera:

- Si la póliza está Abierta, entonces se carga un Gasto Administrativo con un monto de -\$500 (signo negativo) en concepto de “Bonificación Administrativa.”
- Si la póliza está Cerrada, entonces se elimina el Costo Administrativo de mayor valor.
- Si la póliza está Vigente, se eliminan todos sus Gastos Administrativos.

Tareas:

1. Realice el diseño de una solución orientada a objetos con un diagrama UML de clases.
2. Si utiliza patrones de diseño indique cuáles y también indique los participantes (roles) de esos patrones en su solución según el libro de Gamma et al.
3. Programe en Java el modelo anterior completo.
4. Realice un test de unidad para verificar que la carga de una bonificación se aplica efectivamente en una Póliza Vigente.

Programación con Objetos 2 - UNQ
1er. Semestre 2022 - Parcial 1ra. Fecha - Parte teórica

1. ¿Cuáles son las 4 secciones de un test de unidad? Solo escriba el nombre de cada sección (en orden).

A: _____

B: _____

C: _____

D: _____

2. Conteste verdadero o falso. ¿Un mismo objeto puede tener distintos tipos durante su ciclo de vida?

3. Conteste verdadero o falso. Las clases se pueden usar para tipar variables o parámetros. Las interfaces solo para tipar parámetro pero no variables.

4. Seleccione la opción correcta. En la implementación del patrón Observer según Gamma, el Observable sabe:

A: De qué clase concreta son instancias sus observadores

B: Qué harán concretamente sus observadores al ser notificados

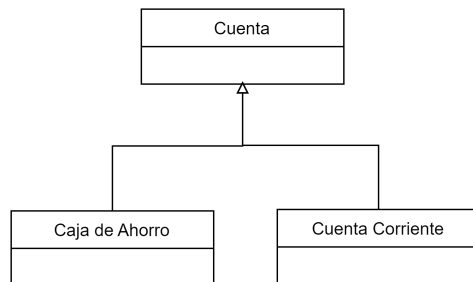
C: Ambas

D: Ninguna de las anteriores

5. Conteste verdadero o falso. No es necesario que la jerarquía de clases del Strategy sea polimórfica.

6. Conteste verdadero o falso. Al igual que en el Strategy, las subclases de un State nunca necesitan instanciar a sus clases hermanas.

7. Conteste verdadero o falso. Dada la siguiente jerarquía y el siguiente código en un lenguaje estáticamente tipado



```
CA ca = new CA();
Cuenta c = ca
```

¿c y ca son instancias diferentes? _____

8. ¿En qué patrón, al ser instanciado se forma una estructura de árbol?

A: Template method

B: Adapter

C: Composite

D: State

E: Strategy

Criterios para ejercicio práctico

- UML
 - (por limitación de tiempo). No es necesario escribir getters y setters. Tampoco constructores. Esto será evaluado en el tp final.
- Generalidades de la implementación
 - Alcanza con escribir solo un getter y un setter para mostrar que saben implementarlos. El resto se asume que existen (y por ende los pueden usar) aunque no es necesario implementarlos.
- Mail del cliente
 - Aceptamos una clase cliente que tenga nombre, apellido y mail o que la póliza tenga el mail del cliente como atributo.
- Completitud de la implementación
 - Para aprobar no es necesario tener todos los métodos que varíen con el state implementados pero sí al menos una funcionalidad donde muestre que sabe implementar el state.

Los puntos que están en rosa, si no los resolvieron bien los deja muy cerca del desaprobado (independientemente del puntaje).

UML	Cardinalidades y variables en las relaciones? Sentido de las relaciones. Las relaciones son correctas?
Póliza	Inventario y gastos administrativo como colecciones
Póliza	Mail del cliente.
Póliza	Implementa correctamente MontoAsegurado, precio (debe multiplicar por 0.075 y no 0.75), gastosAdministrativos
Póliza	Implementa los métodos agregarItem, cerrarInventario, pagar, cancelar, aplicarDescuento en la clase Póliza y delega en el estado.
Póliza	State Los métodos del state deben recibir la póliza como parámetro para poder pedirle el cambio de estado en los métodos que correspondan. Otra opción es que envíen la Póliza cuando crean el estado (me gusta más la anterior) Otra, más alejada de lo ideal, es devolver el estado nuevo (que podría ser el mismo si no cambia) pero es una solución rebuscada. Si no implementan ninguna de estas soluciones probablemente porque no tienen forma de hacer el cambi de estado.
Póliza	No debe haber un strategy en la solución
Póliza	State. Implementa los métodos por defecto como vacío (en una superclase abstracta o con métodos default en las interfaces) para evitar replicar los vacíos en las subclases.
Póliza	State Lógica de cada concrete class para los métodos que correspondan
Póliza	agregarItem devuelve una Póliza para que se pueda seguir usando en la misma línea de código (para responder a “Es una operación que se espera que se invoque varias veces”) .

	Es un detalle para llegar al 10.
Bonificaciones	Póliza debe tener una variable tipada con BonificacionService. Este es un punto importante.
Bonificaciones	La solución ideal es que implementen el bonificar en Póliza (o en un colaborador Bonificador). Por ende los pasos no conforman un template method. Solo se delega al estado el paso de descuento. Una opción alternativa pero alejada de la ideal es que deleguen el bonificar entero en el estado y ahí si conforman un template method. No es la mejor solución porque la responsabilidad de implementar la bonificación no es del estado sino de la Póliza (o de un colaborador)
Test	Test